“Digital Library Search System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

TEJ PRATAP SINGH :2520030476

B. ARYAN :2520030343

Under the Guidance of

Dr. V. Sireesha

Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled "Digital Library Search System" is a record of Bonafide work of TEJ PRATAP SINGH – 2520030476 and B. ARYAN – 2520030343 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

TEJ PRATAP SINGH – 2520030476

B. ARYAN – 2520030343

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled "Digital Library Search System" is a bonafide work done and submitted by TEJ PRATAP SINGH – 2520030476 and B. ARYAN – 2520030343 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, during the academic year 2026-27.

Signature of the Guide

(Dr. V. Sireesha)

Signature of the HOD

(Dr. P. Venkateswara Rao)

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. V. Sireesha, Professor, Department of Computer Science and Engineering, for her continuous guidance, technical suggestions, and academic support throughout the development of the Digital Library Search System. Her direction helped us connect the theoretical concepts of Data Structures and Algorithms - 3 with a practical and demonstrable software system, and encouraged us to maintain accuracy in the implementation of the course outcomes.

We also extend our respectful thanks to Dr. P. Venkateswara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the academic environment and departmental support required to carry out this project. We acknowledge the faculty members of the department for the discussions, review comments, and evaluation feedback that helped us refine the project from an initial search-oriented prototype into a complete Java-based digital library with text-file persistence, a responsive web interface, and a handout-aligned CO1-CO6 Algorithm Lab.

Finally, we thank K L Deemed to be University, Hyderabad Campus, for providing the course structure and project-based learning framework under which this work was completed. We are also grateful to our classmates and peers whose feedback during demonstrations helped us improve the usability, presentation flow, and technical explanation of the system.

TEJ PRATAP SINGH – 2520030476

B. ARYAN – 2520030343

Abstract

Modern academic libraries contain increasingly large collections of books, journals, magazines, research papers, and other metadata-rich resources. When such collections are handled only through manual browsing or simple linear search, users may spend unnecessary time locating a relevant resource, while students studying advanced algorithms receive little visibility into how the underlying search and optimisation logic actually works. The Digital Library Search System was developed to address both concerns: it provides a practical library-search application and simultaneously serves as a Data Structures and Algorithms - 3 project in which the major course outcomes are implemented and demonstrated on a common repository.

The completed system is a Java-based web application operating on 50,000 deterministic library metadata records. The repository contains four equally represented resource types - books, journals, research papers, and magazines - and supports search using title, author, identifier/ISBN, category, subject, and keywords. The project deliberately avoids a database management system. Structured UTF-8 text files remain the persistent source of truth, while Java File I/O loads records into project-owned in-memory structures. This design keeps the data-structure and algorithmic operations visible instead of delegating search to a database engine. The backend uses Java 17+ and the JDK built-in HttpServer, while the frontend is implemented using HTML5, CSS3, and Vanilla JavaScript.

The system integrates the DSA-3 course outcomes within the same application. CO1 classifies the problem signature and selects an appropriate algorithmic family. CO2 implements Naive Pattern Matching, KMP with LPS, Z-Function, Rabin-Karp with double rolling hash, Aho-Corasick, Suffix Array, and Kasai LCP. CO3 includes Wagner-Fischer/Levenshtein, Damerau-Levenshtein, weighted edit distance, Needleman-Wunsch and Smith-Waterman as Module-3 applications, together with the formal advanced dynamic-programming patterns of Interval DP, Bitmask DP, DP on Trees, and DP on Subsets/SOS DP. CO4 implements Ford-Fulkerson, Edmonds-Karp, Dinic, residual networks, max-flow/min-cut, and bipartite matching. CO5 demonstrates P/NP/NP-complete/NP-hard concepts, the canonical reduction chain, Vertex Cover 2-approximation, and

Greedy Set Cover with exact small-instance baselines. CO6 implements Randomized QuickSort, Miller-Rabin, universal/randomized hashing, Reservoir Sampling, parallel search/count, parallel reduce, Blelloch-style prefix sum, and work-span analysis.

The final application includes Home, Explore, Advanced Search, Resource Details, Algorithm Lab, Analytics, and Admin modules. Verification performed on the final build reports 17 Java test groups passed with zero failures, successful repository and CRUD checks, successful CO1-CO6 API smoke tests, and a validated dataset containing 50,000 rows with 50,000 unique resource IDs and 50,000 unique identifiers. Runtime measurements displayed by the application are intentionally treated as machine-dependent measurements rather than fixed performance claims. Overall, the project demonstrates how a large metadata repository can be searched, analysed, and managed using manually implemented data structures and advanced algorithms while retaining a clear, presentation-ready web interface.

Table of Contents

Contents	Page No.
Acknowledgement	
Abstract	
Table of Contents	
List of Figures	
1 Introduction	1
1.1 Overview	1
1.2 Problem Statement	4
1.3 Scope of the Project	6
1.4 Objectives	8
2 System Requirements	10
2.1 Functional Requirements	10
2.2 Non-Functional Requirements	13
3 System Design	18
3.1 High-level Architecture Diagram	18
3.2 Description of Each Layer	20
4 Implementation	25
4.1 Module-wise Implementation	25
4.2 Front-end Logic (UI Rendering, State Management)	28
4.3 API Integration	30
4.4 Data Structures and Algorithm Implementation	32
5 Results	40
6 Challenges & Limitations	47
6.1 Issues Faced During Development	47
6.2 Solutions Applied	48
6.3 Limitations of the System	49
7 Conclusion & Future Work	52
7.1 Conclusion	52

7.2 Future Work	53
7.3 Skills Learned During Development	55
References	57

List of Figures

Figure	Page No.
Fig. 3.1 High-Level Architecture of the Digital Library Search System	18
Fig. 3.2 Search and Algorithm Execution Pipeline	19
Fig. 3.3 Distribution of the 50,000-Record Repository	20
Fig. 5.1 Home Page Interface	42
Fig. 5.2 Advanced Search Interface	42
Fig. 5.3 CO1-CO6 Algorithm Lab Interface	43
Fig. 5.4 Analytics Interface	43
Fig. 5.5 Sample CO2 String Search Benchmark	44
Fig. 5.6 Final Verification Snapshot	45

1. INTRODUCTION

1.1 Overview

A digital library is not simply a web page containing a list of books. It is an information-retrieval environment in which resource metadata must be represented consistently, searched efficiently, and presented in a form that allows a user to move from a broad query to a specific resource without unnecessary delay. Academic repositories become more challenging as they grow because a user may remember only part of a title, an author name, an ISBN-like identifier, a subject, or a keyword. A useful system therefore requires several retrieval strategies rather than one hard-coded search rule.

The Digital Library Search System was developed as a project for Data Structures and Algorithms - 3 (25CS2103E). The project uses a realistic library domain to demonstrate advanced algorithms in a single integrated application. The normal user-facing portion of the system behaves like a searchable digital catalog, while the Algorithm Lab exposes the course-outcome algorithms, their inputs, their outputs, and their complexity-related information for academic demonstration. This separation allows the project to remain useful as a library application while still providing clear evidence of the DSA-3 syllabus.

The final system contains 50,000 deterministic metadata records covering BOOK, JOURNAL, RESEARCH_PAPER, and MAGAZINE resources. Each record contains an internal ID, resource type, title, author, identifier, category, subject, publication year, keywords, availability, and publisher. The repository therefore supports search beyond a single field. A user can search by title when the title is known, by author when only an author is remembered, by identifier for exact retrieval, or by category, subject, and keywords when the requirement is broader.

A central design decision was to avoid a database management system. The project uses structured UTF-8 text files as persistent storage, with data loaded through Java File I/O into in-memory objects and custom structures. This decision is deliberate rather than accidental. Because the course focuses on algorithms, delegating filtering and searching to SQL or another database engine would hide

much of the logic that the project is intended to demonstrate. Keeping persistence simple allows the backend to show where and how indexes, pattern-matching algorithms, dynamic programming, network flow, approximation, randomization, and parallel processing are applied.

The software is organized as a lightweight web application. The frontend is implemented using HTML5, CSS3, and Vanilla JavaScript. The backend is implemented in Java 17+ and uses the JDK built-in HttpServer to expose JSON endpoints and serve the static frontend. No Node.js backend, Maven, Gradle, JDBC driver, MySQL, MongoDB, SQLite, Firebase, or XAMPP is required for normal execution. On Windows, the project can be started from the project root using run.bat; on Linux or macOS, run.sh provides the corresponding workflow.

The project contains two complementary algorithmic layers. The first layer supports normal library operations such as exact identifier lookup, browsing, multi-field search, fuzzy search, analytics, and admin CRUD. The second layer is the CO1-CO6 Algorithm Lab. CO1 acts as the decision layer, identifying the problem signature and selecting the suitable advanced strategy. CO2 handles large-scale string processing. CO3 demonstrates advanced dynamic-programming patterns. CO4 models capacity-constrained assignment with network flow. CO5 demonstrates NP-hardness concepts and approximation. CO6 demonstrates randomized and parallel algorithms. All these demonstrations are tied to the same digital-library domain instead of being presented as unrelated console programs.

From a software-engineering perspective, the project is intentionally modular. The backend separates model classes, storage classes, custom structures, algorithm implementations, service orchestration, and HTTP/JSON handling. The frontend separates page structure, visual styling, reusable API calls, page-specific rendering logic, and lightweight motion effects. The persistent data files are kept outside the compiled classes so that they can be inspected directly during evaluation. This organization makes the project easier to explain file-by-file during a viva and reduces the risk of mixing presentation code with the algorithm engine.

The project also places emphasis on verifiability. The final build contains a dependency-free Java test harness and additional repository, frontend, and dataset

checks. The documented verification state reports 17 Java test groups passed with zero failures. The dataset validation confirms exactly 50,000 rows, 50,000 unique resource IDs, 50,000 unique identifiers, and the presence of all four resource types. Smoke checks cover search, fuzzy search, CO1-CO6 algorithm endpoints, Admin create/read/delete behavior, and the seven main web pages. These checks do not prove that the software is production-ready, but they provide concrete evidence that the implemented academic features execute consistently in the final build.

The importance of the project lies in the connection between theory and implementation. KMP, Z-Function, Rabin-Karp, suffix structures, dynamic programming, max-flow, approximation, randomized algorithms, and parallel primitives are often taught independently. In this project, they are connected to one repository and exposed through one application. As a result, the project becomes both a working digital-library prototype and a practical case study in advanced algorithm selection, implementation, measurement, and explanation.

Background and Motivation

Modern users expect search systems to respond even when the input is incomplete or imperfect. In a library context, a strict exact-match system can fail in common situations: the user may omit a word from a title, type an author name with a minor spelling error, enter only part of an identifier, or search for a conceptual keyword rather than an exact title. A large repository also makes manual browsing impractical. These realities motivate the use of multiple retrieval strategies and a consistent metadata representation.

The academic motivation is equally important. A DSA project should make the selected data structures and algorithms visible. The Digital Library domain provides a natural setting for this. String algorithms can process titles and keywords; edit distance can support typo-tolerant retrieval; suffix structures can demonstrate repeated-substring and similarity-related processing; flow networks can represent assignment problems; approximation algorithms can model hard selection problems; and randomized/parallel methods can demonstrate scalable processing. The domain is therefore broad enough to support the course outcomes without changing the project into several unrelated mini-projects.

Project Context and Course Alignment

The project was refined against the official DSA-3 handout. The final mapping distinguishes the formal course outcome from additional module demonstrations. For example, fuzzy search based on edit distance is a useful Module-3 application, but formal CO3 is demonstrated through Interval DP, Bitmask DP, DP on Trees, and DP on Subsets. Similarly, randomized processing in CO6 is not limited to one algorithm: the project differentiates Las Vegas-style randomized QuickSort from Monte Carlo Miller-Rabin and also includes reservoir sampling, randomized hashing, parallel reduce, prefix sum, and work-span analysis.

This distinction improves academic accuracy during evaluation. It prevents the project from claiming that one convenient feature alone satisfies an entire course outcome. Instead, the normal library pages demonstrate practical search and management, while the Algorithm Lab exposes the broader set of syllabus-aligned algorithms required by the course.

1.2 Problem Statement

Large digital collections are difficult to navigate efficiently when the system relies only on manual browsing or a single basic search method. The problem becomes more significant when users need to search using different metadata fields or when their query contains only a partial phrase or spelling error. A simple linear scan can be used as a baseline, but it does not demonstrate the range of algorithmic strategies expected in an advanced DSA project and may not scale gracefully as the repository increases.

The technical problem addressed by this project is therefore to design and implement a Digital Library Search System that stores a large structured repository, supports multiple search fields, exposes advanced algorithms rather than hiding them behind a DBMS, provides typo-tolerant and algorithm-selectable search, demonstrates the DSA-3 course outcomes CO1 to CO6 within the same domain, and remains straightforward to execute during a faculty demonstration.

A second challenge is representation. Since the project intentionally avoids a DBMS, the system must persist data in a way that is readable, deterministic, and safe enough for academic CRUD operations. The text-file schema must preserve

all resource fields, while the Java backend must reconstruct objects and indexes at startup. Admin operations must modify the persistent source of truth rather than updating only temporary in-memory state.

A third challenge is correctness of course mapping. The project should not merely list algorithm names in the interface. Each algorithm must have a defined input, purpose, and output. String algorithms should actually process patterns; advanced DP should contain the required DP patterns; flow algorithms should operate on capacities and residual graphs; approximation should provide a valid approximate solution and baseline for small instances; randomized algorithms should use randomized choices; and parallel features should compare equivalent sequential and parallel computations without promising a fixed speedup.

Finally, the project must remain explainable. The evaluator should be able to trace a user action from the browser to the Java HTTP endpoint, service layer, selected algorithm, repository or data structure, and returned JSON result. This traceability is a core requirement because the project is both software and an academic demonstration.

Formal Problem Statement

To design and implement a Java-based Digital Library Search System for a 50,000-record metadata repository that supports multi-field and typo-tolerant search, uses structured text files as persistent storage, manually implements the advanced data structures and algorithms required by DSA-3 CO1 to CO6, provides a responsive web interface for normal use and algorithm demonstration, and verifies the correctness of search, CRUD, dataset, API, and course-outcome features without relying on an external DBMS.

Primary Technical Challenges

- Creating a large deterministic dataset that remains easy to regenerate and validate.
- Loading and managing text-file data without delegating search to a database engine.
- Implementing multiple string-search algorithms and returning comparable metrics.

- Supporting fuzzy search while keeping the formal CO3 mapping accurate.
- Modelling an appropriate digital-library assignment problem as a network-flow graph.
- Demonstrating approximation with a provable guarantee instead of claiming universal optimality.
- Demonstrating randomized and parallel algorithms while clearly stating that timing results depend on the execution environment.
- Maintaining a clean boundary between frontend rendering and backend algorithm execution.
- Ensuring that Admin CRUD modifies persistent storage and that temporary verification records are removed after testing.

1.3 Scope of the Project

The scope of the Digital Library Search System is intentionally defined around searchable metadata and advanced algorithm demonstrations. The project is not a full institutional library-management platform and does not attempt to reproduce every function of a commercial integrated library system. Instead, it focuses on the part of the problem that best supports the DSA-3 learning objectives: repository organization, retrieval, algorithm selection, advanced searching, optimization demonstrations, analytics, and file-based persistence.

The data scope includes 50,000 synthetic metadata records. The repository is designed for academic testing and demonstration; it does not contain copyrighted full-text books or research papers. Each resource is represented through metadata fields sufficient for search and filtering. The generator can also create 10K, 25K, 50K, and 100K datasets for scalability experiments, although the final bundled/verified state is 50,000 records.

The functional scope covers Home, Explore, Advanced Search, Resource Details, Algorithm Lab, Analytics, and Admin pages. The Advanced Search page allows the user to select a field and an algorithm. The Algorithm Lab contains the explicit CO1-CO6 demonstrations. Analytics displays repository statistics and measured algorithm timings. Admin CRUD supports add, update, and delete operations that persist to the structured text repository.

The algorithmic scope includes the project-owned custom structures and the algorithms implemented in the final backend. The string-algorithm package includes Naive Pattern Matching, KMP, Z-Function, Rabin-Karp with double rolling hash, Aho-Corasick, Suffix Array, and Kasai LCP. The dynamic-programming package includes edit-distance and sequence-alignment methods together with the formal advanced DP patterns. The flow package includes Ford-Fulkerson, Edmonds-Karp, Dinic, min-cut reachability, and bipartite matching reduction. The approximation package includes Vertex Cover 2-approximation and Set Cover demonstrations. The randomized/parallel package includes randomized QuickSort, Miller-Rabin, randomized hashing, Reservoir Sampling, parallel matching/counting, parallel reduce, and prefix sum.

The deployment scope is local execution. The final project is designed to run on a laptop with JDK 17+ and a modern browser. It is not presented as a cloud-hosted production system, and it does not include institutional authentication, real borrowing transactions, payment services, or external catalog synchronization. Those capabilities are treated as future work rather than existing features.

Included in Scope

- 50,000-record metadata repository with deterministic resource generation.
- Four resource types: books, journals, research papers, and magazines.
- Structured UTF-8 text-file persistence with Java File I/O.
- Custom DynamicArray, singly linked list, queue, stack, and string hash table.
- Multi-field search by title, author, identifier, category, subject, and keywords.
- Naive, KMP, Z-Function, Rabin-Karp, fuzzy, and hash-based search modes.
- Resource browsing, resource details, search history, analytics, and Admin CRUD.
- Handout-aligned CO1-CO6 Algorithm Lab.
- Dependency-free Java verification harness and supporting dataset/frontend checks.
- Windows and Linux/macOS run scripts.

Outside the Current Scope

- Production-grade user authentication and authorization.

- Real institutional student/faculty accounts and single sign-on.
- Cloud deployment and multi-server distributed indexing.
- Full-text storage and indexing of copyrighted books or papers.
- Real borrowing, fine payment, reservation, or notification workflows.
- External publisher, DOI, ISBN, or institutional catalog synchronization.
- Production-level security hardening, rate limiting, audit compliance, and backup orchestration.

1.4 Objectives

The project objectives were defined to satisfy both the functional needs of a digital-library prototype and the academic requirements of the DSA-3 course. The objectives are deliberately measurable: the repository size is fixed for the final build, the supported pages and APIs are identifiable, the course-outcome mapping is explicit, and the final verification report records the checks that were performed.

Objective	Description
Centralized repository	Maintain an organized 50,000-record metadata repository using structured UTF-8 text files as the only persistent data source.
Efficient retrieval	Support search across title, author, identifier/ISBN, category, subject, and keywords using appropriate search algorithms.
Algorithm visibility	Expose the selected algorithm, complexity information, and measured runtime behavior so the DSA logic remains visible during evaluation.
CO1-CO6 integration	Implement and demonstrate the six DSA-3 course outcomes within one coherent Digital Library application.
Fuzzy retrieval	Allow misspelled or approximate queries to return close resources using edit-distance-based dynamic programming.
Custom data structures	Use project-owned dynamic array, list, stack, queue, and hash-table structures where relevant to the engine.
File-based CRUD	Allow administrators to add, update, and delete resources while persisting the changes to resources.txt.
Responsive web interface	Provide an interface that supports normal library use and a separate faculty-oriented Algorithm Lab and Analytics view.
Verification	Test custom structures, algorithms, repository operations, APIs, frontend contracts, and dataset integrity.

Simple execution	Allow the entire system to start using JDK 17+ and a project run script without requiring Node.js, Maven, Gradle, or a DBMS.

2. SYSTEM REQUIREMENTS

System requirements define what the application must do and the quality constraints under which it should operate. In this project, the requirements were derived from the Digital Library use case, the DSA-3 course outcomes, the decision to use text-file persistence, and the need to present the system live during evaluation. The requirements therefore cover both user-facing behavior and academic algorithm behavior.

2.1 Functional Requirements

Functional requirements describe the services that must be observable in the final system. The requirements below correspond to implemented pages, backend services, API endpoints, repository operations, and Algorithm Lab features. They intentionally avoid features such as authentication or real circulation management because those are outside the current project scope.

ID	Requirement	Description
FR-01	Repository Loading	The system shall load structured resource records from data/resources.txt at startup and represent them as Java Resource objects.
FR-02	Dataset Recovery	If the resource file is missing or empty, the system shall be able to generate the configured deterministic dataset using DatasetGenerator.
FR-03	Repository Statistics	The system shall provide total resource count, accessible/available counts, resource-type counts, category counts, storage type, and database status.
FR-04	Browse Resources	Users shall be able to browse repository resources with pagination, resource-type filtering, and sorting.
FR-05	Resource Details	Users shall be able to open an individual resource and view all stored metadata fields.
FR-06	Multi-field Search	Users shall be able to search across all fields or individually by title, author, identifier, category, subject, keywords, or internal ID.
FR-07	Algorithm Selection	Advanced Search shall support Automatic, KMP, Z-Function, Rabin-Karp, Naive, Fuzzy/Edit-Distance, and Hash modes where applicable.
FR-08	Suggestions	The backend shall provide lightweight title/author suggestions for a partial user query.

FR-09	Search Metrics	Search responses shall include algorithm name, measured elapsed time, scanned records, total matches, returned matches, dataset size, and asymptotic-complexity description.
FR-10	Search History	The system shall append search events to search_history.txt and expose recent history through an API.
FR-11	Admin Add	Admin shall be able to create a new resource with validation for required fields and duplicate ID/identifier checks.
FR-12	Admin Update	Admin shall be able to update an existing resource and persist the changed record.
FR-13	Admin Delete	Admin shall be able to delete an existing resource and persist the removal.
FR-14	Activity Logging	Admin mutations and malformed-row events shall be recorded in activity_log.txt.
FR-15	CO1 Classification	Algorithm Lab shall classify a problem hint and select an appropriate strategy with reason and cost information.
FR-16	CO2 String Algorithms	Algorithm Lab shall execute Naive, KMP, Z, Rabin-Karp, Aho-Corasick, Suffix Array, and LCP-related demonstrations.
FR-17	CO3 Advanced DP	Algorithm Lab shall expose Interval DP, Bitmask DP, DP on Trees, DP on Subsets, and the implemented edit/alignment demonstrations.
FR-18	CO4 Network Flow	Algorithm Lab shall execute Ford-Fulkerson, Edmonds-Karp, Dinic, min-cut reachability, and bipartite matching on a library-domain assignment model.
FR-19	CO5 Approximation	Algorithm Lab shall expose P/NP/NP-complete/NP-hard concepts, the reduction chain, Vertex Cover 2-approximation, validity checks, and small-instance exact baselines.
FR-20	CO6 Randomized/Parallel	Algorithm Lab shall demonstrate Randomized QuickSort, Miller-Rabin, randomized hashing, Reservoir Sampling, parallel counting, reduce, prefix sum, and work-span explanation.
FR-21	Analytics	The Analytics page shall visualize current repository statistics and machine-dependent algorithm measurements.
FR-22	Static Frontend Delivery	The Java HTTP server shall serve the frontend pages and JSON APIs from one runnable project.
FR-23	Cross-platform Scripts	The project shall provide Windows and Linux/macOS compile, run, dataset-generation, and test scripts.
FR-24	Verification	The project shall include automated checks for algorithms, custom structures, repository behavior, APIs, frontend assets, and dataset integrity.

Input Requirements

Search input is accepted as text through the web interface. The selected field determines which portion of a Resource record is evaluated. For an all-fields search, title, author, identifier, category, subject, keywords, and publisher are combined into a searchable string. Identifier or ID searches can use the custom hash index for exact retrieval. Fuzzy queries use edit-distance logic rather than requiring exact substring equality.

Admin input must include the required identifying fields. The final backend validates resource ID, type, title, author, and identifier at the HTTP layer, while the Resource model also rejects records that do not contain the minimum identifying values. The text schema removes the vertical-bar delimiter and line breaks from inserted field values so that persisted records remain parseable.

Output Requirements

Normal search returns resource cards and a metrics summary. Resource details return the complete metadata object. Algorithm Lab endpoints return structured JSON that includes the course outcome, title, dataset size, algorithm-specific outputs, and relevant explanatory values. Admin operations return a success message and the affected resource or ID. Error conditions return JSON with a meaningful error message and an appropriate HTTP status such as 400, 404, 405, or 500.

2.2 Non-Functional Requirements

Non-functional requirements define constraints related to performance, portability, maintainability, usability, transparency, and reliability. Because this is an academic prototype, the requirements are focused on repeatable local execution and clear demonstration rather than enterprise service-level agreements.

ID	Quality Attribute	Requirement
NFR-01	Portability	The project should run with JDK 17+ on commonly used desktop operating systems. Windows run.bat and Linux/macOS run.sh are provided.
NFR-02	Dependency Simplicity	Normal execution must not require Node.js, npm, Maven, Gradle, MySQL, MongoDB, SQLite, Firebase, XAMPP, or JDBC.

NFR-03	Readability	Persistent data should remain human-readable through structured UTF-8 text files.
NFR-04	Algorithm Transparency	The project should report the algorithm used and expose DSA behavior rather than hiding retrieval inside a DBMS.
NFR-05	Consistency	The repository should preserve unique IDs and unique identifiers, and malformed rows should not silently corrupt the in-memory dataset.
NFR-06	Responsiveness	The frontend should adapt to desktop and mobile-width layouts and preserve readable navigation and resource cards.
NFR-07	Usability	Normal library operations and the Algorithm Lab should be separated clearly enough for both end-user and evaluator workflows.
NFR-08	Maintainability	Backend responsibilities should be separated into model, structure, algorithm, storage, service, and HTTP packages.
NFR-09	Testability	Core structures, algorithms, storage, CRUD, APIs, and dataset integrity should be testable without external services.
NFR-10	Measurement Integrity	Performance results should be presented as measured values from the current machine; the project must not promise a universal fixed speedup.
NFR-11	Data Safety	Repository rewrites should use a temporary file and replacement operation so an admin update does not intentionally write partial records.
NFR-12	Academic Integrity	The system should accurately distinguish formal CO requirements from additional module demonstrations and avoid claiming unimplemented features.

Hardware Requirements

Component	Requirement
Processor	Intel Core i3 class or equivalent minimum for local execution; additional cores benefit the parallel demonstrations.
Memory	4 GB RAM minimum for the intended local academic workflow; more memory provides additional headroom for IDE/browser use.
Storage	Sufficient free storage for the source code, generated 50,000-record repository, compiled classes, logs, and documentation.
Display	Standard laptop/desktop display capable of running a modern browser.
Network	No external network is required for the core local project after dependencies/JDK are installed.

Software Requirements

Software	Use in Project
Java	JDK 17 or newer; java and javac must be available.
Backend Server	JDK built-in com.sun.net.httpserver.HttpServer.
Frontend	HTML5, CSS3, Vanilla JavaScript.
Persistent Storage	Structured UTF-8 .txt files.
Browser	Any modern browser capable of Fetch API, CSS Grid/Flexbox, and standard ES JavaScript used by the interface.
Editor/IDE	VS Code, IntelliJ IDEA, Eclipse, or another Java-capable editor; not required at runtime.
Version Control	Git/GitHub for source management and submission.
Operating System	Windows 10/11, Linux, or macOS compatible with JDK 17+.
Testing	Dependency-free Java test harness plus Python-based static/dataset checks included in scripts.

Data Requirements

The final data repository is synthetic and deterministic. This is appropriate for an academic algorithm project because it avoids distributing copyrighted content and allows the same metadata structure to be regenerated for testing. The final validated state contains 50,000 rows and exactly four resource types. The generator supports smaller and larger sizes for controlled experiments. Each persistent resource record contains eleven fields separated by the vertical-bar character.

Resource Record Format

```
id|type|title|author|identifier|category|subject|year|keywords|availa
bility|publisher
```

Field	Purpose
id	Internal unique resource key such as LIB000001.
type	BOOK, JOURNAL, RESEARCH_PAPER, or MAGAZINE.

title	Display title of the resource.
author	Author or creator metadata.
identifier	ISBN-like, ISSN-like, DOI-like, or magazine identifier generated by resource type.
category	Broader grouping such as Computer Science, Engineering, Mathematics, Technology, Science, Research, or Computing.
subject	Primary topic such as Algorithms, Machine Learning, Operating Systems, etc.
year	Publication year stored as an integer.
keywords	Comma-separated searchable terms.
availability	Available, Issued, or Digital depending on resource type/state.
publisher	Publisher metadata.

3. SYSTEM DESIGN

The system design follows a layered architecture that keeps presentation, HTTP handling, services, algorithms, custom structures, and persistence logically separate. This separation is important because the project has two audiences: a normal user who expects a clear web interface and an evaluator who needs to trace the implementation of advanced algorithms. The design therefore avoids embedding algorithm logic in frontend JavaScript and avoids hiding data operations behind an external DBMS.

3.1 High-level Architecture Diagram

The high-level architecture is shown in Fig. 3.1. A web browser presents the interface and sends HTTP requests to the Java server. The Java HTTP layer parses the request and delegates to LibraryService or AlgorithmLabService. LibraryService coordinates normal repository functions such as browsing, search, statistics, history, and CRUD. AlgorithmLabService coordinates CO1-CO6 demonstrations. Both services call project-owned algorithms or structures as required. Persistent data remains in text files and is loaded into Resource objects and indexes at startup.

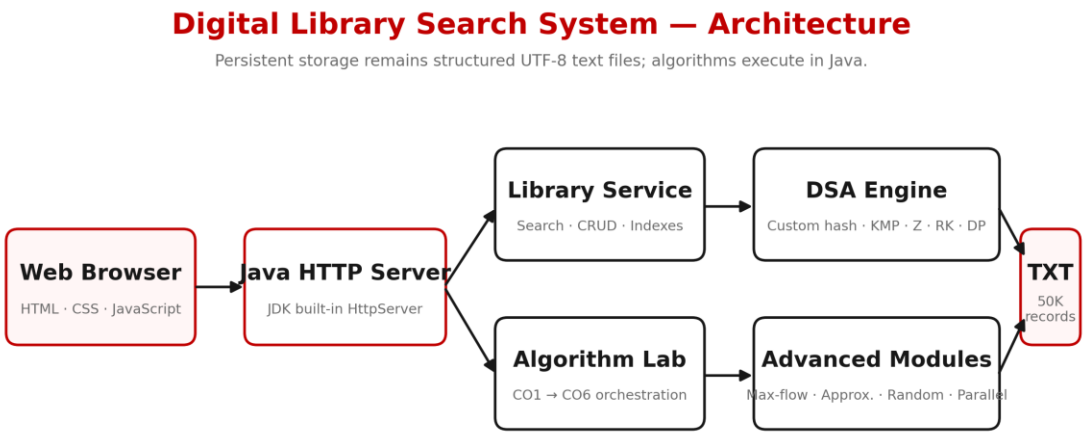


Fig. 3.1 High-Level Architecture of the Digital Library Search System

The architecture intentionally keeps the persistent layer simple. resources.txt is the authoritative resource store, while users.txt, requests.txt, search_history.txt,

and activity_log.txt support auxiliary data. Since the project is not a DBMS project, text-file persistence allows the evaluator to inspect the raw records and then observe how Java loads and processes them.

Search and Algorithm Execution Flow

A normal search begins with a query entered on the Advanced Search page. JavaScript constructs query parameters and sends a GET request to /api/search. The backend determines the selected field and algorithm. If Automatic mode is selected, the service chooses a default strategy based on the field; exact identifier/ID requests are suitable for hash lookup, while general substring search uses KMP in the current normal-search dispatcher. The selected algorithm processes the relevant metadata string, matching resources are collected up to the response limit, and timing/scan/match metrics are returned with the result list.

Algorithm Lab requests follow a similar transport path but execute course-outcome-specific demonstrations. The CO1 endpoint takes a problem hint and classifies the problem signature. The remaining endpoints execute the corresponding algorithm family and return a structured explanation together with outputs. Fig. 3.2 summarizes this query-to-execution flow.

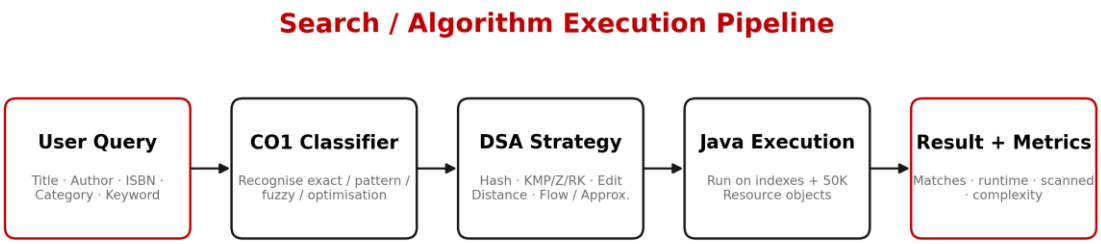


Fig. 3.2 Search and Algorithm Execution Pipeline

Repository Distribution

The final repository contains 50,000 records and distributes the four resource types evenly. Because the generator assigns type cyclically, the verified 50,000-record dataset contains 12,500 books, 12,500 journals, 12,500 research papers, and 12,500 magazines. This balanced distribution avoids a trivial benchmark

dominated by one record type and provides a consistent base for browse and search demonstrations.

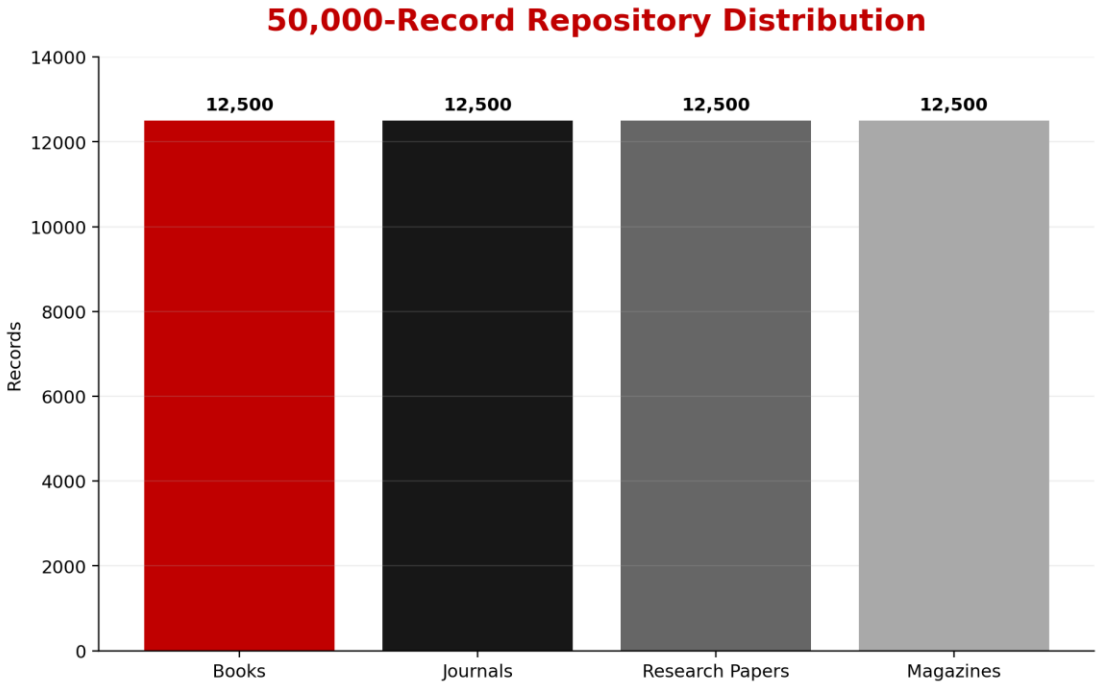


Fig. 3.3 Distribution of the 50,000-Record Repository

3.2 Description of Each Layer

Presentation Layer

The presentation layer is implemented using static HTML pages, a common stylesheet, and page-specific JavaScript. The pages are Home, Explore, Advanced Search, Resource Details, Algorithm Lab, Analytics, and Admin. A shared API helper wraps the Fetch API so page code can request JSON without duplicating request/error-handling logic. The frontend never implements KMP, max-flow, or the other academic algorithms itself; it only sends inputs and renders backend outputs. This ensures the DSA implementation remains in Java, where it can be inspected directly during evaluation.

HTTP and JSON Layer

ApiServer uses the JDK built-in HttpServer. It listens on the configured port, serves static files for non-API paths, and routes /api/ requests to the corresponding handler. Query strings are decoded, request methods are validated, and JSON responses are generated using the project JSON utility. The server also adds

CORS-related headers and no-cache behavior appropriate for the local demo. Using the built-in server keeps the project lightweight and removes the need for a separate application framework.

Service Layer

LibraryService is the main orchestration component for normal library operations. It loads resources through TextRepository, builds indexes, supports paging and sorting, dispatches search modes, records search history, and performs add/update/delete operations. AlgorithmLabService is the orchestration component for the course outcomes. It constructs representative inputs, calls the corresponding algorithm classes, measures selected operations, and packages outputs in a form suitable for JSON rendering.

The separation between services and algorithms is important. The algorithm classes do not know about HTML pages or HTTP responses. They receive strings, arrays, matrices, graphs, or other compact inputs and return algorithmic outputs. This makes the core logic easier to test and keeps the educational implementation independent of presentation concerns.

Algorithm Engine

The algorithm engine is divided into classes for string algorithms, suffix structures, dynamic programming, network flow, approximation, and randomized/parallel algorithms. The final handout-aligned build avoids using Java collection classes inside the algorithm package as substitutes for the required algorithms. Custom queue/data structures are used where suitable, while Java concurrency primitives are used only for the genuine parallel demonstrations. This design follows the course intent that students build the algorithms rather than invoke library implementations.

Custom Data-Structure Layer

Structure	Role
DynamicArray<T>	Resizable array used by project services/repository logic; supports add, insert, get, set, remove, iteration.

SinglyLinkedList<T>	Hand-built linked-list structure supporting add, addFirst, indexed traversal, and removal.
ArrayQueue<T>	Circular-array queue used by BFS-style logic including Edmonds-Karp/Dinic support and Aho-Corasick failure-link construction.
ArrayStack<T>	Stack abstraction built on DynamicArray.
StringHashTable<V>	Project-owned chained hash table used for exact resource ID and identifier indexing.

Persistence Layer

TextRepository owns the file-oriented persistence operations. It ensures that required files exist, reads resources line by line, converts valid rows into Resource objects, logs malformed records instead of crashing the complete startup, and rewrites resources through a temporary file when Admin operations modify the repository. The persistence layer therefore remains transparent and directly inspectable.

The DatasetGenerator creates deterministic metadata records and supports different dataset sizes. The final verified state is 50,000 records. The generator is also used by tests to create smaller temporary repositories so that service and API behavior can be verified without modifying the final dataset.

Layer Interaction Summary

Layer / Component	Responsibility
Web Browser	Captures user input and renders resource cards, metrics, algorithm JSON, analytics, and admin controls.
ApiServer / JsonUtil	Receives HTTP requests, validates methods/parameters, calls services, and serializes responses.
LibraryService	Normal library orchestration: load, index, browse, search, stats, history, CRUD.
AlgorithmLabService	CO1-CO6 problem construction, algorithm dispatch, explanation, and measured demonstrations.
Algorithms	Hand-built implementations for strings, suffixes, DP, flow, approximation, randomized and parallel algorithms.
Structures	Custom dynamic array, list, queue, stack, and hash table.

TextRepository / DatasetGenerator	Persistent text I/O, safe rewrites, logs, and synthetic dataset generation.
data/*.txt	Persistent source of truth and auxiliary logs/history.

Backend Package Structure

Package / File	Purpose
digitallibrary.Main	Application bootstrap; resolves paths, ensures dataset, creates repository/services/server.
digitallibrary.algorithms	String, suffix, advanced DP, flow, approximation, randomized, and parallel logic.
digitallibrary.http	HttpServer routing and JSON utility.
digitallibrary.model	Resource and SearchResult models.
digitallibrary.service	LibraryService and AlgorithmLabService orchestration.
digitallibrary.storage	TextRepository and DatasetGenerator.
digitallibrary.structures	Project-owned data structures.
backend/test/digitallibrary	Dependency-free test harness.

Design Rationale

The architecture favors explicitness over infrastructure complexity. A database-backed framework could provide persistence and querying with fewer lines of code, but it would reduce visibility into the data structures and algorithms. A large Java framework could provide routing, dependency injection, and serialization, but it would also increase setup requirements. The chosen architecture uses standard JDK facilities and project-owned logic so the evaluator can follow the complete request path with minimal external abstraction.

The architecture also reduces presentation risk. The project can be demonstrated offline once the JDK is available. Because the frontend is served by the same Java process, there is no need to start a separate Node development server. Because persistence is file-based, there is no need to configure database credentials or ensure that a database service is running. This directly addresses

common live-demo failures such as unavailable ports, missing package installations, or disconnected database services.

4. IMPLEMENTATION

Implementation converts the design into executable modules. The final project uses plain Java compilation, static frontend files, and structured text persistence. This chapter follows the template requirement to explain the implementation with pseudocode and then connects each pseudocode block to the actual modules used in the project.

4.1 Module-wise Implementation

Application Bootstrap Module

Main.java is the entry point. It resolves the project root, data directory, and frontend directory. It creates the data directory if necessary, checks whether resources.txt exists and contains data, generates 50,000 records when the repository is missing or empty, constructs TextRepository, LibraryService, AlgorithmLabService, and ApiServer, and then starts the server. The port defaults to 8080 but can be changed through the run script or command-line argument.

Pseudocode 4.1 - Application Startup

```
ALGORITHM StartApplication(port)
    root <- current working directory
    dataDir <- root / "data"
    frontendDir <- root / "frontend"
    ENSURE dataDir exists

    IF resources.txt does not exist OR is empty THEN
        GenerateDataset(resources.txt, 50000, deterministicSeed)
    END IF

    repository <- new TextRepository(dataDir)
    libraryService <- new LibraryService(repository)
    algorithmLab <- new AlgorithmLabService(libraryService)
    server <- new ApiServer(port, libraryService, algorithmLab,
frontendDir)
    server.start()
END ALGORITHM
```

Repository and Dataset Module

TextRepository reads resources.txt line by line. Each line is split into exactly eleven fields and converted to a Resource object. Malformed rows are skipped and logged. When CRUD modifies the repository, the full in-memory resource

sequence is written to a temporary file, then the temporary file replaces resources.txt. DatasetGenerator produces deterministic records using a fixed seed and predictable resource-type cycling.

Pseudocode 4.2 - Load Repository

```
ALGORITHM LoadResources(file)
    resources <- empty DynamicArray
    FOR each line in file DO
        IF line is blank THEN CONTINUE
        TRY
            resource <- ParsePipeSeparatedResource(line)
            resources.add(resource)
        CATCH malformedRecord
            Append activity log entry
        END TRY
    END FOR
    RETURN resources
END ALGORITHM
```

Library Service Module

LibraryService rebuilds two custom hash indexes after repository load: one maps lower-cased internal IDs to Resource objects and the other maps lower-cased identifiers to Resource objects. These indexes support expected near-constant-time exact lookup. Paging and filtering operate on the in-memory repository. Search dispatch converts the selected metadata field to a searchable text string and then invokes the chosen matching strategy.

Pseudocode 4.3 - Search Dispatcher

```
ALGORITHM Search(query, field, algorithm, limit)
    VALIDATE query
    q <- lowercase(trim(query))

    IF algorithm = AUTO THEN
        IF field is ID or IDENTIFIER THEN algorithm <- HASH
        ELSE algorithm <- KMP
    END IF

    startTimer()
    IF algorithm = HASH AND field is exact-key field THEN
        result <- lookup in custom hash index
        scanned <- 1
    ELSE
        FOR each resource in repository DO
            scanned <- scanned + 1
            text <- SelectFieldText(resource, field)
            matched <- RunSelectedAlgorithm(text, q, algorithm)
            IF matched THEN
```

```
                totalMatches <- totalMatches + 1
                IF returnedResults < limit THEN append resource
            END IF
        END FOR
    END IF

    elapsed <- stopTimer()
    Append search-history record
    RETURN results + metrics
END ALGORITHM
```

Admin CRUD Module

Admin CRUD is handled through POST, PUT, and DELETE requests to /api/admin/resource. Before adding a resource, the service rejects a duplicate internal ID or duplicate identifier. Update requires an existing ID and also checks that the replacement identifier does not belong to another record. Delete scans the in-memory resource sequence, removes the matching item, rebuilds indexes, persists the updated repository, and records the operation in activity_log.txt.

Pseudocode 4.4 - Persistent Add/Update/Delete

```
ALGORITHM AddResource(resource)
    IF id already exists OR identifier already exists THEN REJECT
    resources.add(resource)
    RebuildIndexes()
    SaveResourcesUsingTemporaryFile()
    Append activity log "ADD"
END ALGORITHM

ALGORITHM UpdateResource(resource)
    REQUIRE existing resource ID
    REJECT identifier collision with another resource
    replace matching in-memory record
    RebuildIndexes()
    SaveResourcesUsingTemporaryFile()
    Append activity log "UPDATE"
END ALGORITHM

ALGORITHM DeleteResource(id)
    find record by ID in resource sequence
    IF not found THEN return false
    remove record
    RebuildIndexes()
    SaveResourcesUsingTemporaryFile()
    Append activity log "DELETE"
    return true
END ALGORITHM
```

Algorithm Lab Module

AlgorithmLabService provides a faculty-oriented layer over the algorithm classes. It does not replace the normal Digital Library functionality. Instead, it creates representative inputs from the same domain and exposes structured results for each course outcome. This improves presentation because the evaluator can see both the practical library application and the advanced algorithm evidence without switching to separate projects.

Module	Implementation Summary
Home	Live repository statistics, global search entry, featured/recent resources, navigation to core modules.
Explore	Pagination, type filter, sorting, quick transition to advanced search.
Advanced Search	Field selection, algorithm selection, search metrics, result cards.
Resource Details	Complete metadata display and subject-based similar-resource navigation.
Algorithm Lab	CO1-CO6 algorithm execution and JSON-style live output.
Analytics	Repository statistics, CO2 timing comparison, CO6 sequential/parallel metrics.
Admin	Create, update, delete, refresh, and inspect sample repository entries.
Backend Services	Normal repository orchestration and course-outcome orchestration.
Persistence	resources.txt plus users, requests, history, activity logs.
Verification	Java tests, API smoke checks, frontend/project contract checks, dataset validation.

4.2 Front-end Logic (UI Rendering, State Management)

The frontend uses page-level JavaScript rather than a client framework. This keeps the project lightweight and makes the logic easy to inspect. api.js provides the common request methods for statistics, resources, search, suggestions, history, algorithm endpoints, analytics, and admin operations. Each page then contains only the state required for that page.

The Explore page maintains page number, page size, total matching records, selected resource type, sorting field, and sort order. When a filter changes, page state resets to the first page and the frontend requests a fresh page from the

backend. The Advanced Search page reads the query, field, and selected algorithm, sends a request, then renders metric cards and resource cards. The Algorithm Lab maintains the active CO and query/problem hint, maps the CO to the correct endpoint, and displays formatted JSON output. The Analytics page requests one combined analytics response and converts timing values into relative bar widths for visual comparison.

The interface deliberately avoids pretending that the browser executes the DSA algorithms. JavaScript is responsible for input capture, navigation, Fetch API calls, formatting, and DOM rendering. The algorithm work remains in the Java backend, preserving a clear separation of concerns.

Pseudocode 4.5 - Advanced Search UI Flow

```
ON search form submission:
    prevent normal form reload
    q <- query input
    field <- selected metadata field
    algorithm <- selected algorithm
    IF q is empty THEN stop

    response <- GET /api/search(q, field, algorithm, limit)
    metrics <- response.metrics
    resources <- response.resources

    render metric cards
    IF resources is empty THEN render "No results"
    ELSE render one resource card for each returned resource
END
```

Responsive Rendering

The shared stylesheet uses CSS Grid, Flexbox, responsive breakpoints, and a consistent visual system. Desktop layouts use multiple columns for resource cards and analytics cards, while narrower viewports collapse to fewer columns. Navigation links simplify on smaller screens. The interface also respects reduced-motion preferences by disabling nonessential animations when the browser indicates that preference.

Client-side Error Handling

The shared API helper attempts to parse every backend response as JSON and throws an error when the HTTP status is not successful. Page modules catch that

error and display a visible message or toast. This prevents silent failure when a required query parameter is missing, a resource does not exist, a duplicate Admin ID is submitted, or the backend is not running.

4.3 API Integration

The frontend and backend communicate through HTTP using JSON. Most retrieval endpoints use GET because they do not mutate persistent data. Admin creation uses POST, replacement uses PUT, and deletion uses DELETE. The server routes requests by path, checks the HTTP method, parses query parameters or a small JSON object, and returns JSON with an appropriate status code.

Method	Endpoint	Purpose
GET	/api/stats	Repository counts, types, categories, storage/database information.
GET	/api/resources	Paginated/filterable/sortable resource list.
GET	/api/resource?id=...	Single resource details.
GET	/api/search?q=...	Search results and algorithm metrics.
GET	/api/suggest?q=...	Title/author suggestions.
GET	/api/history	Recent search-history lines.
POST	/api/admin/resource	Add a persistent resource.
PUT	/api/admin/resource	Update a persistent resource.
DELETE	/api/admin/resource?id=...	Delete a persistent resource.
GET	/api/algorithms/classify	CO1 problem classification.
GET	/api/algorithms/string	CO2 string/suffix demonstration.
GET	/api/algorithms/dp	CO3 advanced-DP demonstration.
GET/POST	/api/algorithms/flow	CO4 network-flow demonstration.
GET/POST	/api/algorithms/approximation	CO5 approximation demonstration.
GET	/api/algorithms/co6	CO6 randomized and parallel demonstration.
GET	/api/algorithms/fuzzy	Focused fuzzy/DP compatibility endpoint.
GET	/api/algorithms/randomized	Focused randomized compatibility endpoint.
GET	/api/algorithms/parallel	Focused parallel compatibility endpoint.
GET	/api/analytics	Combined repository and timing data for Analytics page.

Pseudocode 4.6 - HTTP Request Routing

```
ALGORITHM HandleHttpRequest(exchange)
    IF method = OPTIONS THEN return CORS preflight response

    path <- request path
    IF path starts with "/api/" THEN
        Parse query parameters
        SELECT path
            CASE /api/search: call LibraryService.search(...)
            CASE /api/algorithms/string: call
AlgorithmLabService.stringDemo(...)
            CASE /api/admin/resource: dispatch POST/PUT/DELETE
            ...
            DEFAULT: return 404 JSON
        END SELECT
    ELSE
        Resolve requested static file under frontend directory
        Reject path traversal or missing file
        Return file with correct content type
    END IF
END ALGORITHM
```

JSON Representation

JsonUtil converts strings, numbers, booleans, maps, iterables, arrays, and Resource objects into JSON. It also parses the flat JSON object used by Admin create/update requests. The implementation is intentionally lightweight because the project does not require nested object binding or an external JSON framework. Resource exposes a map representation containing all metadata fields so the same model can be serialized consistently across endpoints.

HTTP Error Semantics

Status	Use in Project
400 Bad Request	Missing/invalid query parameters, invalid year, missing required admin fields, or duplicate constraints reported as invalid request.
404 Not Found	Unknown API path, missing static page, or requested resource ID not found.
405 Method Not Allowed	Endpoint called with an unsupported HTTP method.
500 Internal Server Error	Unexpected exception during request processing; server logs stack trace and returns a JSON error message.

4.4 Data Structures and Algorithm Implementation

This section describes the DSA implementation that forms the academic core of the project. The algorithms are grouped according to the official course outcomes. The descriptions below emphasize what is implemented, why it is used, and how the Digital Library provides a suitable demonstration context.

CO1 - Problem Classification and Algorithm Selection

CO1 is the decision layer. A system should not apply the same algorithm to every problem. AlgorithmLabService.co1Classification() examines a problem hint and maps recognizable signatures to an appropriate strategy. Exact identifiers map to custom hash lookup; substring search maps to KMP/Z/Rabin-Karp; fuzzy or alignment tasks map to dynamic programming; repeated-substring or similarity tasks map to suffix structures; assignment/capacity tasks map to network flow; NP-hard optimization maps to approximation; primality/sampling maps to randomized algorithms; and reduce/prefix/large-data tasks map to parallel algorithms.

Pseudocode 4.7 - CO1 Strategy Selection

```
ALGORITHM SelectStrategy(problemHint)
    IF hint looks like exact ID/identifier
        RETURN Custom Hash Index
    ELSE IF hint mentions typo/fuzzy/alignment
        RETURN Dynamic Programming
    ELSE IF hint mentions similarity/suffix/repeated substring
        RETURN Suffix Array + LCP
    ELSE IF hint mentions allocation/matching/flow
        RETURN Network Flow
    ELSE IF hint mentions NP-hard/cover/scheduling/optimisation
        RETURN Approximation
    ELSE IF hint mentions prime/random/sample
        RETURN Randomised Algorithms
    ELSE IF hint mentions parallel/prefix/reduce
        RETURN Parallel Algorithms
    ELSE
        RETURN KMP/Z string search
END ALGORITHM
```

CO2 - String Algorithms and Suffix Structures

The normal library search most directly connects to CO2. Naive Pattern Matching is retained as a baseline. KMP computes an LPS array and reuses prefix information after a mismatch instead of restarting the pattern from the beginning.

The Z-Function builds the pattern-plus-text string and identifies positions where the prefix length reaches the pattern length. Rabin-Karp compares polynomial rolling hashes and verifies actual characters when both hash values match. The final implementation uses two moduli to reduce the practical risk of collision.

Aho-Corasick is implemented for multi-pattern search. Patterns are inserted into a hand-built ASCII trie, failure links are constructed with a custom queue, and the text is scanned once while output counts propagate through failure links. Suffix Array is constructed using a doubling approach with a hand-built merge sort, and the LCP array is computed with Kasai's algorithm. These suffix structures support longest-repeated-substring and similarity-oriented demonstrations in the Algorithm Lab.

Pseudocode 4.8 - KMP Search

```
ALGORITHM KMP(text, pattern)
    IF pattern is empty RETURN 0
    lps <- BuildLPS(pattern)
    i <- 0; j <- 0
    WHILE i < length(text)
        IF text[i] = pattern[j]
            i <- i + 1; j <- j + 1
            IF j = length(pattern) RETURN i - j
        ELSE IF j > 0
            j <- lps[j - 1]
        ELSE
            i <- i + 1
    END WHILE
    RETURN -1
END ALGORITHM
```

Pseudocode 4.9 - Rabin-Karp with Double Rolling Hash

```
ALGORITHM RabinKarp(text, pattern)
    IF pattern longer than text RETURN -1
    compute two pattern hashes
    compute two hashes for first text window
    compute highest base powers

    FOR each window position i
        IF both pattern hashes equal both window hashes THEN
            verify actual characters
            IF equal RETURN i
        END IF
        roll both window hashes to next position
    END FOR
    RETURN -1
END ALGORITHM
```

CO3 - Advanced Dynamic Programming

The final build explicitly separates Module-3 edit/alignment applications from the formal CO3 requirements. Wagner-Fischer/Levenshtein computes the minimum insertion, deletion, and substitution cost. Damerau-Levenshtein adds adjacent transposition handling, which is relevant to common typing errors. Weighted edit distance assigns different costs to insert, delete, and substitute operations. Needleman-Wunsch and Smith-Waterman demonstrate global and local alignment respectively.

The formal CO3 patterns are also implemented. Interval DP is demonstrated through Matrix Chain Multiplication and Optimal Binary Search Tree. Bitmask DP is demonstrated through Travelling Salesperson, Hamiltonian Path detection, and a bounded resource-selection problem. DP on Trees is represented through a maximum-weight independent-set computation on a tree. DP on Subsets/SOS DP computes the sum over all submasks for every mask. This broader implementation avoids the inaccurate claim that fuzzy search alone constitutes formal CO3.

Pseudocode 4.10 - Wagner-Fischer Edit Distance

```
ALGORITHM Levenshtein(a, b)
    create matrix d[0..|a|][0..|b|]
    initialize first column with 0..|a|
    initialize first row with 0..|b|

    FOR i from 1 to |a|
        FOR j from 1 to |b|
            substitution <- 0 if a[i-1] = b[j-1] else 1
            d[i][j] <- MIN(
                d[i-1][j] + 1,        // delete
                d[i][j-1] + 1,        // insert
                d[i-1][j-1] + substitution)
        END FOR
    END FOR
    RETURN d[|a|][|b|]
END ALGORITHM
```

Pseudocode 4.11 - Matrix Chain Interval DP

```
ALGORITHM MatrixChainCost(dimensions p)
    n <- number of matrices
    dp[i][i] <- 0 for every matrix i
    FOR chainLength from 2 to n
        FOR each interval [i, j] of this length
            dp[i][j] <- infinity
            FOR split k from i to j-1
```

```
                cost <- dp[i][k] + dp[k+1][j]
                        + p[i] * p[k+1] * p[j+1]
                dp[i][j] <- MIN(dp[i][j], cost)
            END FOR
        END FOR
    END FOR
    RETURN dp[0][n-1]
END ALGORITHM
```

Pseudocode 4.12 - Bitmask TSP

```
ALGORITHM TSP(distance)
    dp[mask][city] <- infinity
    dp[1][0] <- 0                  // start at city 0
    FOR each mask containing start city
        FOR each city u included in mask
            FOR each city v not included in mask
                nextMask <- mask OR (1 << v)
                dp[nextMask][v] <- MIN(dp[nextMask][v],
                                       dp[mask][u] +
distance[u][v])
            END FOR
        END FOR
    END FOR
    RETURN minimum dp[allCities][u] + distance[u][0]
END ALGORITHM
```

CO4 - Network Flow

CO4 models assignment and capacity constraints. The Digital Library demonstration creates a bipartite network with a source, student/request nodes, limited library-resource nodes, and a sink. Source-to-student edges have unit capacity, allowed student-to-resource edges have unit capacity, and resource-to-sink edges represent available capacity. The maximum flow therefore equals the maximum feasible number of assignments under the constructed request graph.

Ford-Fulkerson uses DFS augmenting paths, Edmonds-Karp uses BFS to choose shortest augmenting paths in edge count, and Dinic builds level graphs and blocking flows. All operate on a residual network that allows previous flow decisions to be revised through reverse capacity. After max-flow, reachability from the source in the residual graph identifies the source side of a min-cut. The project also provides a direct bipartite matching reduction that builds the corresponding flow graph automatically.

Pseudocode 4.13 - Edmonds-Karp

```
ALGORITHM EdmondsKarp(capacity, source, sink)
    residual <- copy(capacity)
    maxFlow <- 0

    WHILE BFS finds path source -> sink in residual graph
        augment <- minimum residual capacity on path
        FOR each edge (u, v) on path
            residual[u][v] <- residual[u][v] - augment
            residual[v][u] <- residual[v][u] + augment
        END FOR
        maxFlow <- maxFlow + augment
    END WHILE

    minCutReachable <- vertices reachable from source in residual
graph
    RETURN maxFlow, residual, minCutReachable
END ALGORITHM
```

CO5 - NP-Completeness and Approximation

CO5 contains both conceptual and executable components. The Algorithm Lab explains P, NP, NP-complete, and NP-hard and displays the canonical reduction chain used in the project explanation. For a concrete approximation algorithm, the project implements the standard Vertex Cover 2-approximation based on maximal matching. Whenever an uncovered matching edge is selected, both endpoints are placed in the cover. The returned cover is checked for validity. For small graphs, the project also enumerates exact vertex covers so the observed approximation ratio can be reported. The theoretical guarantee remains the standard bound of at most two times the optimum for this algorithm.

Greedy Set Cover is included as an additional NP-hard optimization demonstration. The algorithm repeatedly chooses the unused set with the largest number of currently uncovered elements. For small instances, an exact baseline enumerates subsets to identify the optimal number of sets. The project does not claim that greedy Set Cover always produces the optimum; its purpose is to show practical approximation behavior and comparison with a small exact instance.

Pseudocode 4.14 - Vertex Cover 2-Approximation

```
ALGORITHM VertexCover2Approx(vertices, edges)
    matchedVertex[v] <- false for all v
    chosen[v] <- false for all v
```

```
    FOR each edge (u, v)
        IF NOT matchedVertex[u] AND NOT matchedVertex[v] THEN
            matchedVertex[u] <- true
            matchedVertex[v] <- true
            chosen[u] <- true
            chosen[v] <- true
        END IF
    END FOR

    RETURN all vertices marked chosen
END ALGORITHM
```

CO6 - Randomized and Parallel Algorithms

CO6 differentiates algorithms that use randomness from algorithms that use multiple processors. Randomized QuickSort uses a randomly selected pivot. The output is always correctly sorted, while the running time depends on the random pivot sequence, making it a Las Vegas-style example. Miller-Rabin tests whether a number is probably prime by selecting random witnesses; repeated rounds reduce error probability, making it a Monte Carlo-style example. Universal/randomized hashing selects parameters from a hash family to reduce sensitivity to adversarial or unlucky key distributions. Reservoir Sampling selects k uniformly distributed indices from a stream of n items using O(k) memory.

The parallel portion uses Java ForkJoin for repository matching/counting and tree-style reduction. A Blelloch-style exclusive prefix sum demonstrates scan/prefix processing. The Algorithm Lab measures sequential and parallel execution of equivalent repository matching and reports the measured speedup. The documentation and UI explicitly state that this speedup depends on processor count, JVM state, workload, scheduling, and overhead; no universal constant speedup is claimed.

Pseudocode 4.15 - Reservoir Sampling

```
ALGORITHM ReservoirSample(n, k)
    reservoir[0..k-1] <- first k stream indices
    FOR i from k to n-1
        j <- uniform random integer in [0, i]
        IF j < k THEN
            reservoir[j] <- i
        END IF
    END FOR
    RETURN reservoir
```

```
END ALGORITHM
```

Pseudocode 4.16 - Parallel Reduce

```
ALGORITHM ParallelReduce(values, lo, hi)
    IF range size <= threshold THEN
        RETURN sequential sum of values[lo..hi)
    END IF

    mid <- (lo + hi) / 2
    fork left task ParallelReduce(values, lo, mid)
    right <- ParallelReduce(values, mid, hi)
    left <- join left task
    RETURN left + right
END ALGORITHM
```

Algorithm Complexity Summary

Algorithm	Complexity	Project Role
Naive Pattern Matching	O(n*m)	Baseline substring search.
KMP	O(n+m)	Prefix/LPS-based pattern search.
Z-Function	O(n+m)	Prefix-match based pattern search.
Rabin-Karp	Average O(n+m)	Rolling-hash pattern search; actual matches verified.
Aho-Corasick	O(text + matches + automaton construction)	Multi-pattern matching.
Kasai LCP	O(n) after suffix array	LCP construction.
Levenshtein	O(n*m)	Typo/fuzzy similarity.
Matrix Chain DP	O(n^3)	Interval DP.
TSP Bitmask DP	O(2^n * n^2)	Subset-state optimization.
Edmonds-Karp	O(VE^2)	Max-flow with BFS augmenting paths.
Vertex Cover 2-Approx	Polynomial	Provable <= 2*OPT cover size.
Reservoir Sampling	O(n) time, O(k) space	Uniform streaming sample.
Randomized QuickSort	Expected O(n log n)	Random-pivot sorting.
Parallel Reduce	O(n) work, logarithmic ideal span	Tree-style parallel primitive.

5. RESULTS

Results were evaluated through the final verification pipeline and through live application behavior. The project does not treat one machine's runtime as a universal benchmark. Instead, the repeatable results are repository integrity, successful test groups, endpoint behavior, page availability, and algorithm-output equivalence where appropriate. Runtime values shown in the interface are measured on the current environment and are presented as observations.

Dataset Verification Results

Verification Item	Observed Result	Status
Total resource rows	50,000	Passed
Unique resource IDs	50,000	Passed
Unique identifiers	50,000	Passed
BOOK records	12,500	Present
JOURNAL records	12,500	Present
RESEARCH_PAPER records	12,500	Present
MAGAZINE records	12,500	Present
Malformed final rows	0 reported by validator	Passed

The equal resource-type distribution is a consequence of the deterministic generator, which cycles through the four types. Uniqueness of both internal IDs and identifiers is important because exact hash lookup and CRUD validation assume that those keys identify one resource unambiguously.

Automated Verification Results

Area	Final Verification Observation
Java test harness	17 test groups passed; 0 failed.
Custom structures	DynamicArray, LinkedList, Queue, Stack, and StringHashTable checks passed.
Text repository	Round-trip persistence and append logging checks passed.
Dataset generation	Temporary dataset generation and uniqueness checks passed.

Library service	Search plus add/update/delete checks passed.
Algorithm Lab schemas	CO1-CO6 result structures checked.
JSON utility	Escaping and flat-object parsing checks passed.
HTTP smoke test	Server startup and /api/stats request passed.
API smoke coverage	Search, fuzzy search, CO1-CO6, and Admin create/read/delete round-trip passed.
Static pages	Home, Explore, Search, Resource Details, Algorithm Lab, Analytics, and Admin checks passed.
Dependency audit	No production DBMS/JDBC/Node/Maven/Gradle requirement in final execution path.

Website Interface Results

The final frontend provides seven linked pages. The Home page introduces the repository and provides a global search entry. Explore presents paginated resource cards and filters. Advanced Search exposes field and algorithm selection and renders measured search metrics. Resource Details displays all metadata for one selected item. Algorithm Lab provides the CO1-CO6 faculty-demonstration interface. Analytics displays live repository statistics and timing comparisons. Admin allows persistent CRUD operations.

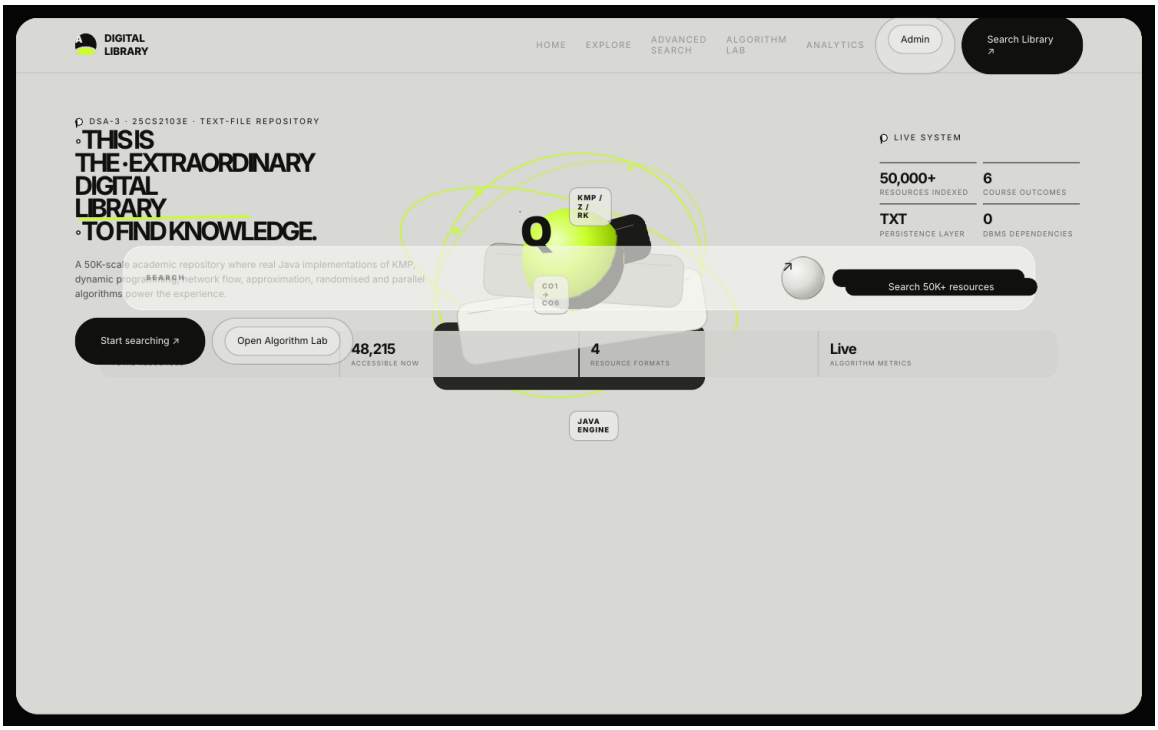


Fig. 5.1 Home Page Interface

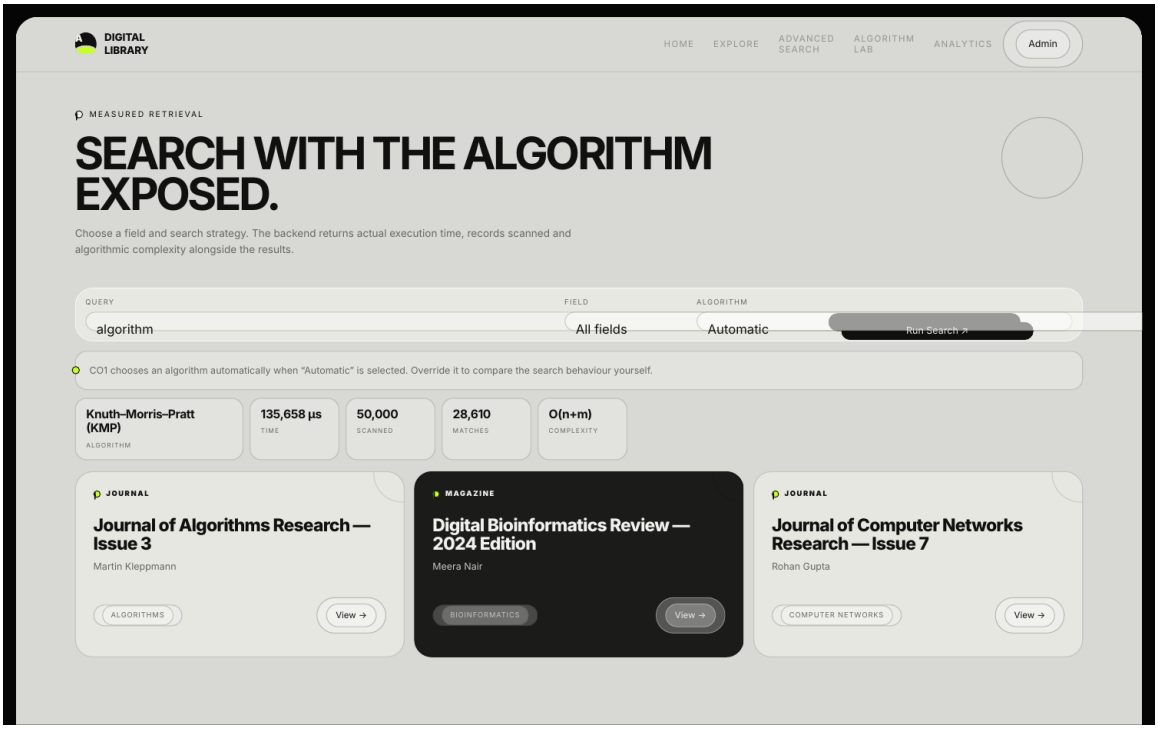


Fig. 5.2 Advanced Search Interface

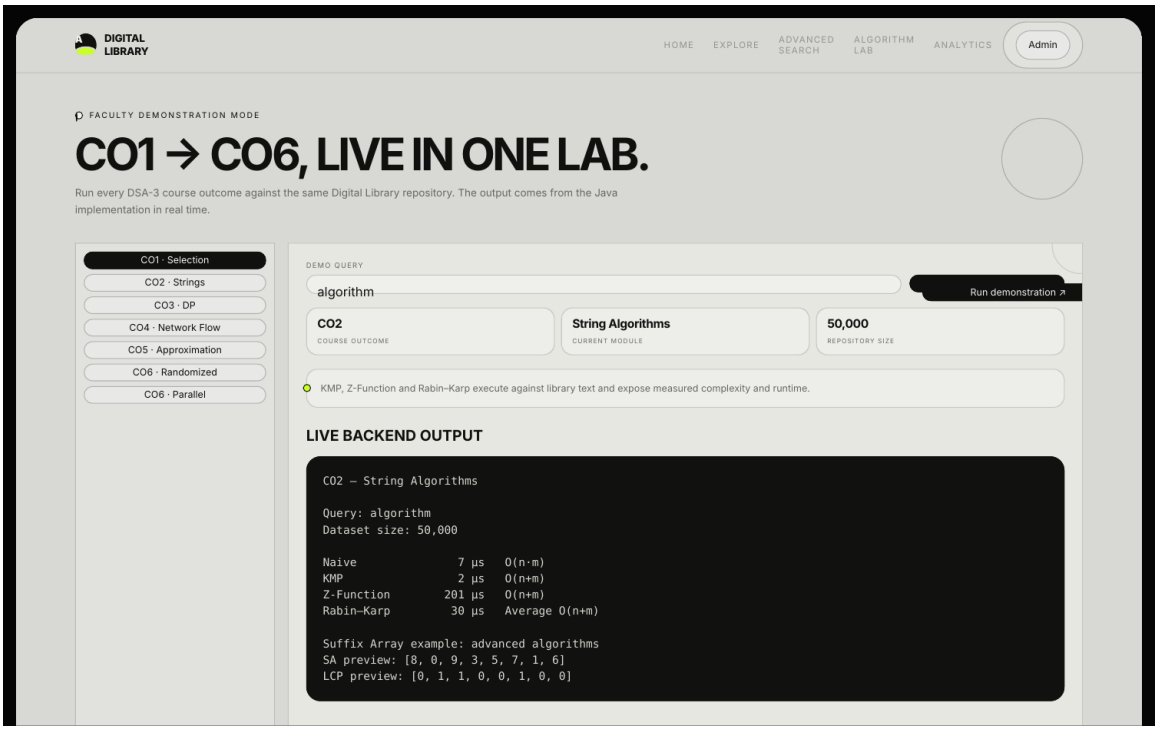


Fig. 5.3 CO1-CO6 Algorithm Lab Interface

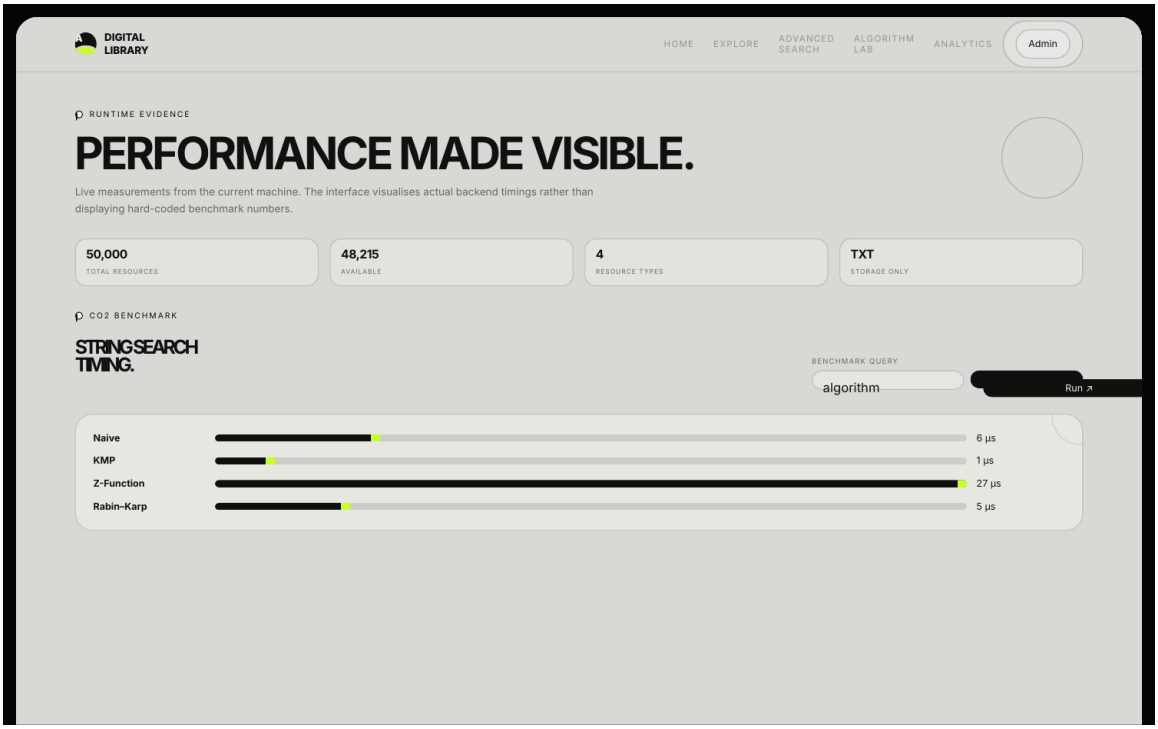


Fig. 5.4 Analytics Interface

Search and Performance Observation

The search engine returns the algorithm name, elapsed microseconds, scanned count, total matches, returned count, dataset size, and complexity description with each search response. These fields make the runtime behavior visible during a demonstration. The project includes Naive, KMP, Z-Function, Rabin-Karp, fuzzy, and hash-based modes. Direct timing comparison must be interpreted carefully because very small operations are affected by JVM warm-up, operating-system scheduling, CPU frequency, cache state, and the specific query.

A previously recorded verification sample for the CO2 benchmark is shown in Fig. 5.5. It is included as an example of the interface output rather than a guaranteed ordering of algorithms. A later run can legitimately produce different microsecond values. The system therefore reports current measurements instead of hard-coding one benchmark result.

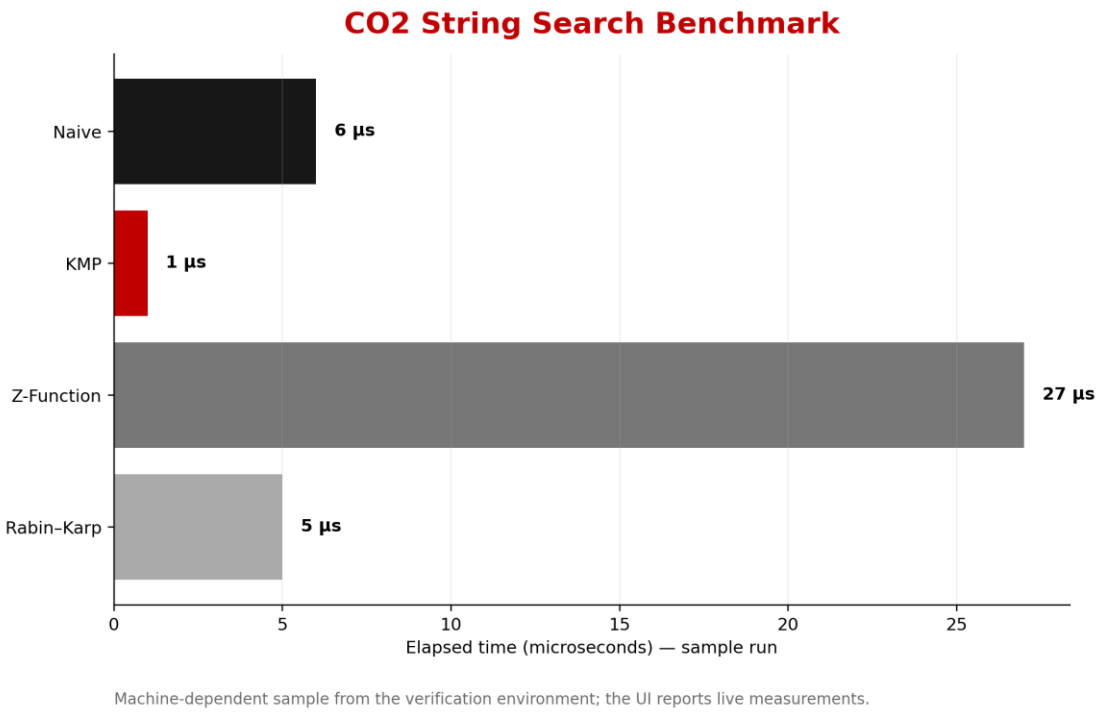


Fig. 5.5 Sample CO2 String Search Benchmark

Course Outcome Demonstration Results

CO	Observed Demonstration
CO1	Problem hints are classified into exact lookup, string matching, DP, suffix/similarity, flow, approximation, randomized, or parallel strategies.
CO2	Naive, KMP, Z, Rabin-Karp, Aho-Corasick, Suffix Array, Kasai LCP, and repeated-substring output are available.
CO3	Edit/alignment results plus Interval DP, Bitmask DP, DP on Trees, and SOS/subset DP outputs are available.
CO4	Ford-Fulkerson, Edmonds-Karp, and Dinic run on the same assignment network; matching and min-cut information are returned.
CO5	Complexity-class explanation, reduction chain, valid Vertex Cover 2-approximation, exact small-instance baseline, and Set Cover demonstration are returned.
CO6	Randomized QuickSort, Miller-Rabin, reservoir sampling, randomized hash bucket, parallel counting, reduce, prefix sum, and measured speedup are returned.

Final Verification Snapshot

Fig. 5.6 summarizes the final verified project state recorded during project preparation. The important repeatable facts are the 50,000-record repository, 50,000 unique IDs, successful Java test groups, seven web pages, absence of a

production DBMS dependency, and complete CO1-CO6 coverage in the documented build.

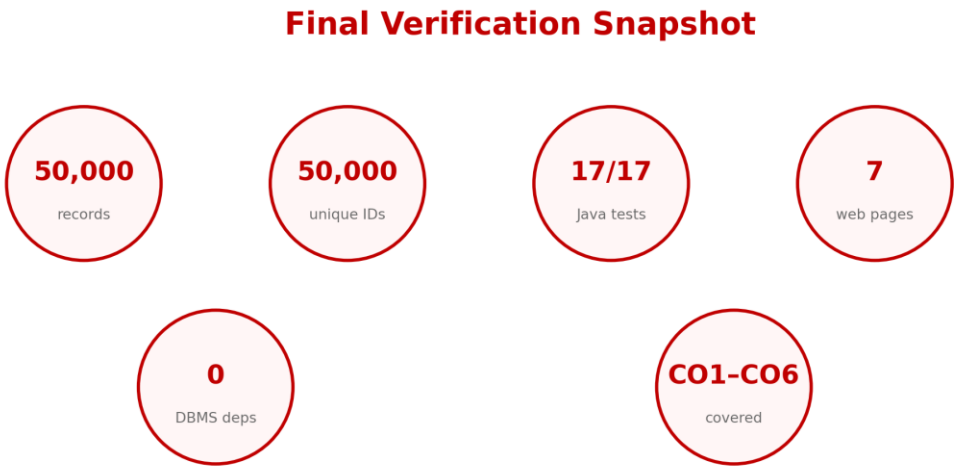


Fig. 5.6 Final Verification Snapshot

Interpretation of Results

The results show that the project achieved its primary academic and functional objectives. The repository can be generated, loaded, searched, browsed, and modified without a DBMS. The API and frontend operate together as one local Java application. Exact, substring, fuzzy, and algorithm-lab operations are exposed through identifiable endpoints. The test harness covers the custom structures and the major advanced algorithms, while the dataset validator verifies the integrity assumptions used by the repository and indexes.

The results also demonstrate an important distinction between correctness and performance. Correctness is verified through known algorithm cases, equivalence checks, valid-cover checks, repository round trips, CRUD, and API behavior. Performance is observed through current timing measurements. This distinction prevents the documentation from presenting a machine-dependent timing sample as a universal scientific result.

The final system is therefore suitable for a B.Tech DSA project demonstration: it contains an application domain, a substantial deterministic dataset, manually implemented advanced algorithms, a web interface, a clear

architecture, persistent CRUD, automated checks, and a direct mapping to the course handout.

6. CHALLENGES & LIMITATIONS

6.1 Issues Faced During Development

The project evolved through several iterations, and a number of practical issues had to be resolved before the final handout-aligned build. Some issues were implementation-related, while others concerned academic accuracy and live-demo reliability.

Issue	Why It Was a Challenge
Keeping the project DSA-focused	An early or conventional implementation could rely on a database query engine for retrieval, but that would hide the core search logic expected in a DSA project.
Text-file persistence with CRUD	Updating a persistent line-oriented repository requires rewriting the resource file while preserving record structure and key uniqueness.
Large repository startup and testing	A 50,000-record final dataset is useful for demonstration but unnecessarily heavy for every unit-style test.
Multiple search algorithms	Different algorithms require different internal state and complexity, but the frontend needs one consistent response shape.
Fuzzy-search interpretation	Edit distance is useful in Module 3 but should not be misrepresented as the complete formal CO3 requirement.
CO mapping accuracy	Network flow, approximation, and randomized/parallel topics must be demonstrated meaningfully without pretending they are ordinary title-search features.
Parallel timing variability	Parallel execution may be faster or slower depending on processor cores, workload size, JVM warm-up, and task overhead.
Live demonstration simplicity	Separate frontend/backend servers, package managers, and databases increase the chance of setup failure during evaluation.
Windows paths and ports	Project paths may contain spaces, and default port 8080 may already be occupied on a development machine.
Dataset integrity after tests	Admin CRUD tests can temporarily modify the repository, so the final packaging process must remove temporary records and restore the expected dataset state.

A further challenge was balancing the normal application with the academic demonstration. If every advanced algorithm were forced directly into ordinary library search, the interface would become artificial and confusing. If the project contained only separate console programs for the course outcomes, it would lose

the coherence of a single application. The final solution was to keep the Digital Library as the main product and add a dedicated Algorithm Lab that reuses the same domain and repository context.

6.2 Solutions Applied

Challenge	Applied Solution
DSA-focused persistence	Structured UTF-8 text files were retained as the permanent source of truth; search and indexing remain Java responsibilities.
Safe resource rewriting	TextRepository writes a temporary resource file and then replaces the target file, reducing the chance of a deliberately partial rewrite.
Small temporary test datasets	DatasetGenerator is reused by tests to create compact temporary repositories while the final project remains 50,000 records.
Unified SearchResult metrics	The service layer returns resources plus a consistent metrics map independent of the selected string/fuzzy/hash strategy.
Formal CO3 separation	Documentation and Algorithm Lab explicitly distinguish edit/alignment applications from Interval/Bitmask/Tree/Subset DP.
Algorithm Lab design	CO4-CO6 are shown as advanced domain extensions instead of being falsely described as mandatory steps for every search.
Measured, not promised, speedup	The UI reports current sequential/parallel timing and an observed speedup value with a machine-dependence warning.
Single-process local execution	Java HttpServer serves both the static frontend and JSON API; no Node server or DBMS service is needed.
Alternative port support	run.bat accepts a port argument, allowing execution on 8090 or another port when 8080 is occupied.
Verification cleanup	Final verification removes temporary admin records, restores exactly 50,000 rows, clears logs/history as appropriate, and recompiles production sources.

Academic Accuracy Improvements

The final handout-accurate revision expanded the project beyond the earlier simplified mapping. Aho-Corasick and Kasai LCP were included in the string/suffix layer. Formal CO3 now contains Interval DP, Bitmask DP, DP on Trees, and DP on Subsets rather than being described only as fuzzy search. CO4 includes Ford-Fulkerson, Edmonds-Karp, Dinic, min-cut, and matching. CO5 uses Vertex Cover 2-approximation with an exact small-instance baseline and validity check. CO6 includes Miller-Rabin, randomized hashing, reservoir sampling,

parallel reduce, prefix scan, and work-span explanation. These changes make the implementation more faithful to the course outcome statements.

Execution Reliability Improvements

The project is packaged so the normal Windows workflow is to open the Project folder and run run.bat. The script compiles the backend and starts the Java application. If resources.txt is empty, startup can generate the dataset automatically. This workflow removes several fragile dependencies that commonly affect live demonstrations. Linux/macOS equivalents are provided through shell scripts. The README documents the required JDK version and the fact that npm start is not part of this project.

6.3 Limitations of the System

The final system is an academic prototype, and its limitations should be stated clearly. These limitations do not invalidate the project objectives; they define the boundary between a course project and a production digital-library platform.

Limitation	Current Boundary
No production authentication	The final web application does not implement secure student/faculty/admin login, password management, or role-based authorization.
Metadata only	The repository stores synthetic metadata rather than copyrighted full-text documents.
Local persistence	Text files are appropriate for the DSA demonstration but are not intended as a concurrent multi-user transactional datastore.
Single-process architecture	The backend runs as one Java process; it is not distributed or horizontally scalable.
No real circulation workflow	Borrowing, return, reservation, due dates, fines, and notifications are outside the current implementation.
No external catalog integration	The project does not call real ISBN, DOI, publisher, or institutional library APIs.
Simple JSON parser	The project JSON utility is intentionally lightweight and handles the flat structures required by current Admin input; it is not a general-purpose JSON framework.
Search ranking	Substring search returns matches but does not implement a sophisticated relevance-ranking model such as BM25 or learning-to-rank.

Synthetic dataset	Metadata is generated for project testing, so repository distributions and titles are not intended to represent a real institutional collection.
Parallel overhead	ForkJoin demonstrations are educational; small inputs may not benefit from parallel execution because scheduling overhead can exceed useful work.
Security hardening	The local academic server is not presented as internet-facing production infrastructure and does not include full security controls such as TLS termination, authentication middleware, rate limiting, and hardened upload policies.
Accessibility depth	The interface includes responsive and reduced-motion considerations, but it has not been claimed to have undergone a formal WCAG compliance audit.

The most important limitation for future engineering work is the persistence model. A structured text repository is useful because it exposes the algorithms and is simple to inspect, but it becomes unsuitable when many users must update the same repository concurrently. A production system would normally require transactional storage, indexing, authentication, backup strategy, and controlled access. Those additions should be treated as deployment evolution rather than retroactively claimed as features of the current project.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

The Digital Library Search System demonstrates that a large metadata repository can be converted into a practical advanced-algorithm project without depending on an external database engine. The final application loads a deterministic 50,000-record repository from structured text files, maintains Java objects and custom indexes in memory, provides multi-field search and fuzzy retrieval, exposes persistent Admin CRUD, and presents the complete system through a responsive web interface.

The project satisfies its academic purpose by integrating the DSA-3 course outcomes into one coherent domain. CO1 evaluates a problem signature and selects an advanced strategy. CO2 implements linear-time string matching and suffix-based processing. CO3 contains the required advanced dynamic-programming patterns and also demonstrates edit distance and sequence alignment. CO4 models assignment and capacity constraints through network flow. CO5 demonstrates complexity-class reasoning and approximation with a provable Vertex Cover bound. CO6 applies randomized and parallel algorithms while reporting runtime measurements carefully.

The final architecture also supports clear evaluation. The browser communicates with a Java HttpServer through JSON APIs. Services separate normal library behavior from course-outcome orchestration. Algorithm classes remain independent of HTTP and presentation logic. The custom structure package makes the project-owned DSA components visible. The text-file repository can be inspected directly, and the verification scripts provide repeatable evidence of repository integrity and feature execution.

The verification state strengthens the conclusion. Seventeen Java test groups passed with zero failures, and the dataset was validated at 50,000 rows with unique IDs and unique identifiers. API smoke checks covered normal search, fuzzy search, the CO1-CO6 endpoints, and an Admin CRUD round-trip. Static page checks covered all seven main pages. These results demonstrate that the final

deliverable is more than a presentation mock-up; it is a runnable Java application with implemented algorithms and persistent repository behavior.

At the same time, the project deliberately does not overstate its scope. The system is not a production institutional library platform, and timing samples are not universal performance claims. The value of the project is the accurate combination of an understandable application domain, a large deterministic dataset, visible advanced algorithms, lightweight local execution, and a clear academic mapping. This makes the Digital Library Search System suitable for a B.Tech CSE project submission and a focused demonstration of advanced data-structure and algorithm concepts.

7.2 Future Work

Future work can extend the project in two directions: production-oriented library functionality and deeper algorithmic research. Any future extension should preserve the current strength of the project - clear algorithm visibility - while introducing infrastructure only where it solves a real requirement.

Future Enhancement	Description
Authentication and Roles	Add secure student, faculty, librarian, and administrator accounts with role-based permissions.
Institutional Catalog Integration	Import real metadata through approved library, DOI, or ISBN services and synchronize updates.
Circulation Workflow	Add borrow, return, reservation, due-date, fine, and notification modules.
Transactional Persistence	Introduce a database for production multi-user updates while keeping the DSA engine available for algorithm demonstrations.
Full-text Indexing	Support permitted full-text documents and build an inverted index for terms and phrase retrieval.
Relevance Ranking	Add term weighting, BM25-style ranking, or other transparent relevance models rather than only boolean matching.
Advanced Similarity	Expand suffix-based similarity into document-to-document comparison and clearly display matched segments or similarity evidence.
Recommendation Layer	Use borrowing/search history or content metadata to recommend related resources, provided the model and evaluation are clearly documented.

Distributed Search	Partition very large repositories across multiple processes or machines and evaluate distributed indexing/search.
Cloud Deployment	Containerize or deploy the application with HTTPS, authentication, backups, monitoring, and controlled configuration.
Mobile Application	Provide a mobile client that consumes the same API.
Accessibility Audit	Perform a formal WCAG-oriented audit and improve keyboard navigation, semantic labeling, contrast, and screen-reader support.
Algorithm Benchmark Suite	Add repeatable warm-up, multiple-trial sampling, confidence summaries, and dataset/query classes for more rigorous performance analysis.
Concurrency and Locking	If text persistence is retained for teaching, add file locks/version checks to demonstrate safe concurrent update concepts.

Recommended Evolution Path

A sensible development sequence would first add authentication and real circulation workflow while preserving the existing metadata search engine. The next step would be to separate teaching persistence from production persistence: the Algorithm Lab could continue operating on project-owned structures, while transactional user operations could use a database. After this separation, relevance ranking, full-text indexing, external catalog integration, and distributed search could be evaluated without weakening the clarity of the current DSA demonstrations.

7.3 Skills Learned During Development

The project required both algorithmic and software-engineering skills. The most important learning outcome was understanding that selecting an algorithm is as important as implementing it. CO1 forced the project to distinguish exact-key lookup, substring search, similarity, dynamic programming, network optimization, NP-hard approximation, randomization, and parallelism as different problem classes.

Implementation of CO2 improved practical understanding of prefix preprocessing, rolling hashes, trie failure links, suffix ordering, and LCP construction. CO3 required reasoning about DP state representation: interval boundaries, subset masks, tree parent/child dependencies, and sum-over-subsets

transitions. CO4 made residual capacity and augmenting paths concrete rather than purely theoretical. CO5 reinforced the difference between exact optimization and approximation guarantees. CO6 showed why randomized correctness models and parallel work/span analysis must be explained separately from simple wall-clock timing.

The project also strengthened Java engineering skills. These included package design, custom generic data structures, File I/O, safe file replacement, HTTP servers, request routing, query parsing, JSON serialization, concurrency through ForkJoin, test harness design, and cross-platform scripting. Frontend work improved understanding of the Fetch API, page-specific state, responsive CSS, error messaging, and the role of a presentation layer that does not duplicate backend logic.

Finally, the project provided experience in technical presentation and documentation. The code had to be explainable to an evaluator, the course-outcome mapping had to match the official handout, performance claims had to remain honest, and the final repository had to be easy to run. These skills are directly relevant to larger software projects where correctness, maintainability, traceability, and communication are required together.

Skill Area	Learning Outcome
Advanced algorithm selection	Recognizing whether a task is string search, DP, flow, NP-hard optimization, randomized, or parallel.
String algorithms	KMP, Z, Rabin-Karp, Aho-Corasick, suffix arrays, LCP.
Dynamic programming	Edit/alignment DP, interval DP, bitmask DP, tree DP, subset/SOS DP.
Graph algorithms	Residual networks, Ford-Fulkerson, Edmonds-Karp, Dinic, matching, min-cut.
Approximation	Vertex Cover 2-approximation, exact small-instance baselines, Set Cover behavior.
Randomized/parallel computing	Randomized QuickSort, Miller-Rabin, hashing, reservoir sampling, ForkJoin reduce/prefix concepts.
Java backend development	HttpServer, File I/O, service separation, JSON endpoints, exceptions.
Frontend development	HTML, CSS, Vanilla JavaScript, Fetch API, responsive rendering.

Testing and verification	Unit-style algorithm tests, smoke tests, dataset validation, dependency audit.
Documentation and viva preparation	Architecture explanation, course mapping, honest performance interpretation, reproducible run instructions.

REFERENCES

[1] K L Deemed to be University, “Data Structures and Algorithms - 3 (25CS2103E), Course Handout, Academic Year 2026-2027.”

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

[3] J. Kleinberg and É. Tardos, Algorithm Design, Pearson, 2005.

[4] J. Erickson, Algorithms, University of Illinois, 2019.

[5] A. Laaksonen, Competitive Programmer’s Handbook, University of Helsinki, 2018.

[6] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, Network Flows: Theory, Algorithms, and Applications, Pearson, 1993.

[7] V. V. Vazirani, Approximation Algorithms, Springer, 2003.

[8] R. Motwani and P. Raghavan, Randomized Algorithms, Cambridge University Press, 1995.

[9] Oracle, “Java Platform, Standard Edition 17 API Specification,” Java SE Documentation.

[10] Oracle/OpenJDK, “com.sun.net.httpserver HttpServer API,” JDK HTTP Server Documentation.

[11] MDN Web Docs, “Fetch API,” used for browser-to-backend HTTP integration concepts.

[12] MDN Web Docs, “HTML and CSS Reference,” used for responsive web-interface implementation concepts.

[13] Project repository documentation: README.md, HANDOUT_CO_MAPPING.md, FACULTY_DEMO_GUIDE.md, and VERIFICATION.md from the final Digital Library Search System build.